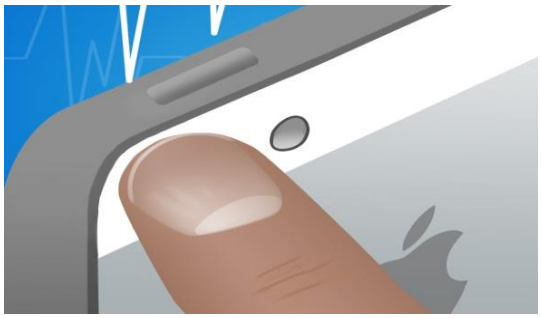
Heartbeat Report

1. Project overview

Our project deals with developing an iOS app that’s able to measure heart rate by reading color changes on arterial pressure change. It generally requires you to place your finger over the flash led and camera.



The operative term here is photoplethysmograph. It's the term for what your phone is acting as.
Basically, your blood vessels rhythmically dilate and contract with every heart beat. This is measurable as either a change in reflectance or transmittance (depending on how the photoplethysmograph is configured).
In the case of your phone, it is measuring reflectance, as the light is adjacent to the camera.

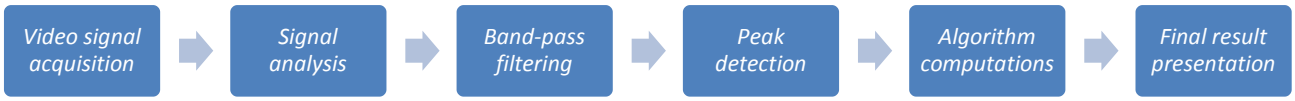
In medical instruments, it's generally done with infrared light, because blood absorbs shortwave infrared more than red or white light, and your tissue is more transparent to infrared. However, there is no reason it wouldn't work with a phone camera.

It is time to open your photo app and observe what happens when you cover the camera with your finger. The frame is not black but slightly red. The ambient light is traveling through your finger tissues and is reaching the camera sensor. Despite the image looks like an almost static red frame,



The periodic oscillations of the blood flow in the vessels produce weak brightness variations that are undetectable for the naked eye, but can be discovered with signal processing.

2. How does it work?

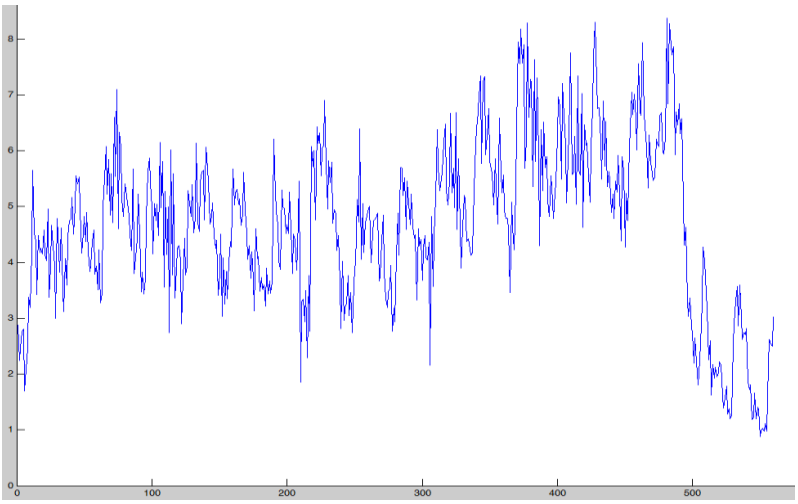


2.1 Video signal acquisition

The video recorded on the iPhone camera at 30 FPS (frames per second) and in low resolution to obtain maximum performance. The led flashlight is required to obtain maximum light reflectance from the finger.

2.2 Signal analysis

Every frame recorded is converted to a single float value which is an average of all the pixels in the green plane. As a result we get an unfiltered signal:



2.3 Band-pass filtering

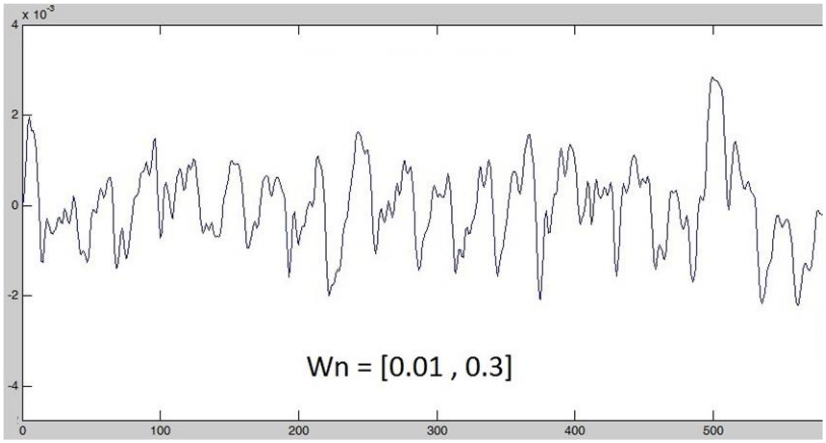
A band-pass filter attenuates frequencies outside the interest band. This reduces the noise in later processing steps (peak fine-tuning) and makes the resulting heart rate signal smoother. In our case, a third-order Butterworth filter is used. The cutoff frequencies have been set to contain our band of interest: 45-180 bpm (beats per minute).

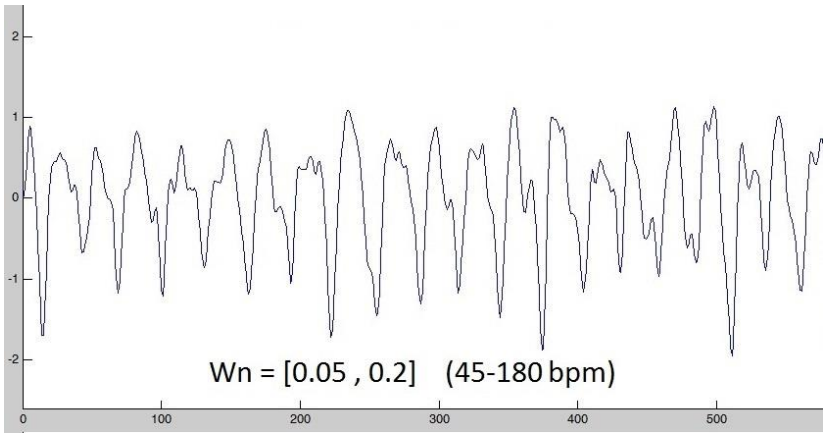
The boundary parameter *Wn* (normalized cutoff frequency) given to the Butterworth filter is calculated by this formula:

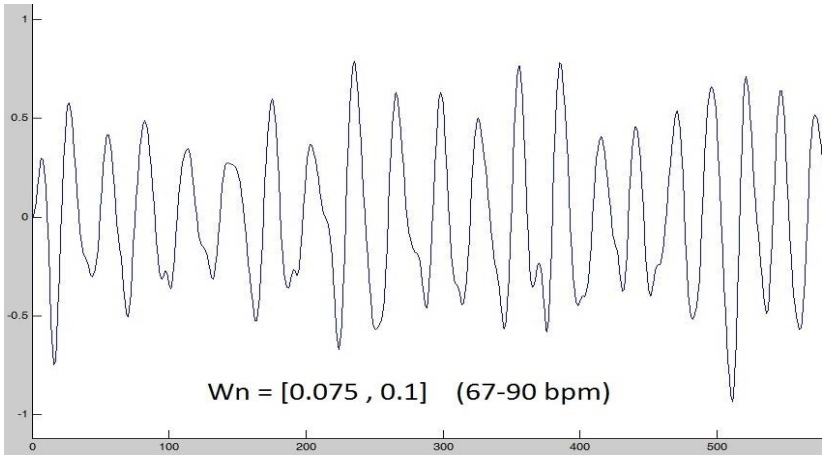
$$\frac{BPM}{\frac{FPS}{2} \cdot 60}$$

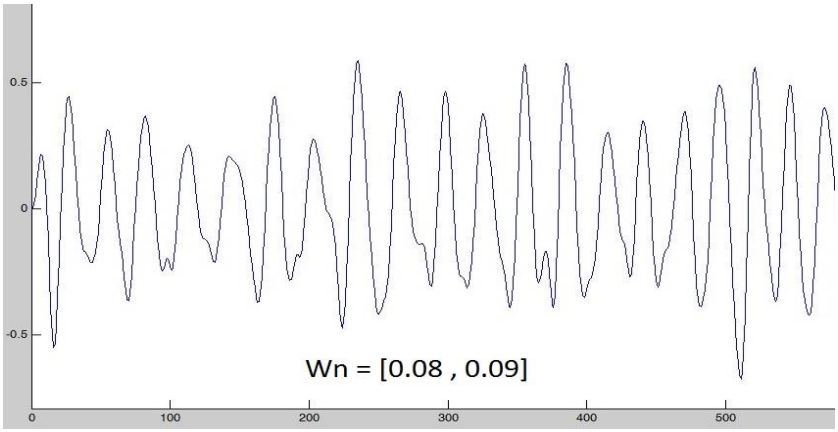
Another important parameter given to the Butterworth filter is *n* – the order of a digital Butterworth filter. We experimented and applied the Butterworth filter (on the unfiltered processed signal) with different values for the *Wn* and *n* parameters. We shall presents the results now:

Different boundaries

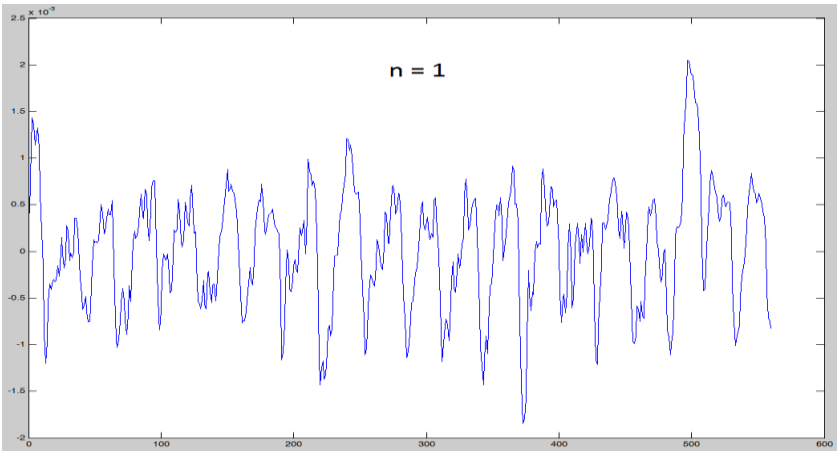


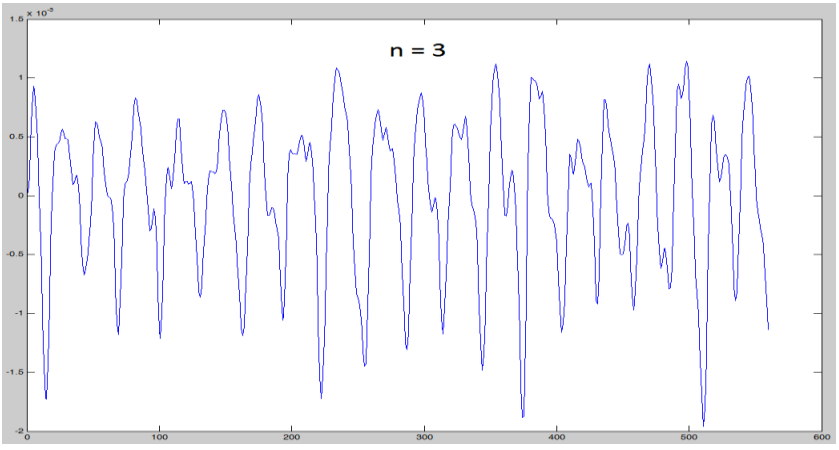


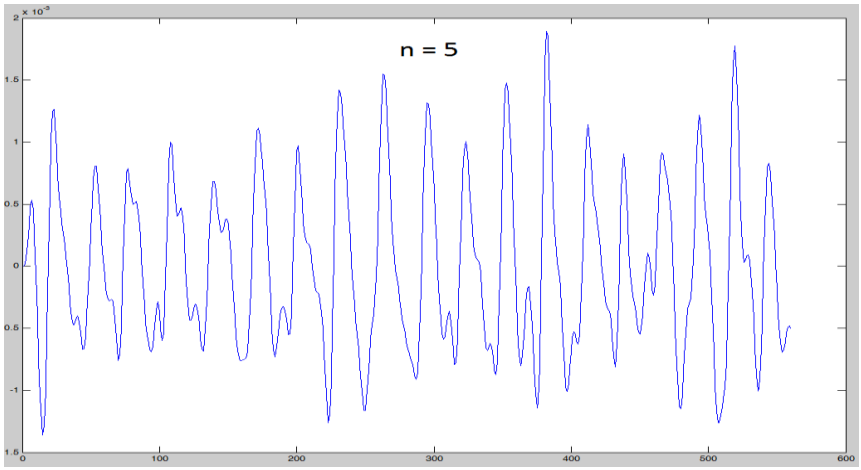




Different order







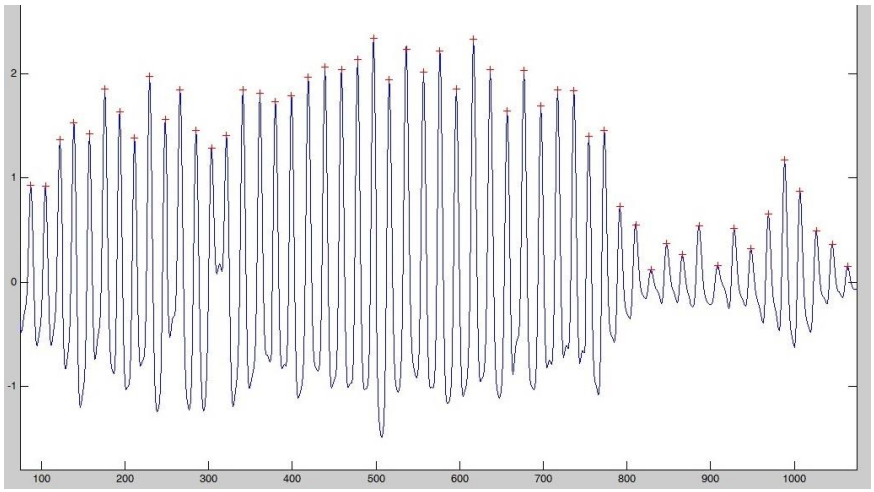
In Our algorithm we are using *Wn = [0.05, 0.2] and n = 3.* Other filter types can be tested. We chose Butterworth because:

- It is an IIR filter and the order required for a given bandwidth is much lower than with a FIR filter. Lower order usually means less computations.
- It has flat pass-band and stop-bands compared to other IIR structures that show ripples. This avoids favoring certain frequencies over others in the valid range.

2.4 Peak detection

Once the Butterworth filtering is done for the current sliding window contents, magnitude peaks in the interest band are spotted thanks to the simple, straight-forward algorithm. A sample is taken as a peak, given a window, if it is the highest in that window.

We present an example result of running the *isPeak* algorithm on a processed filtered signal graph. The peaks found by the algorithm are marked with a red *+*.



2.5 Algorithm computations

The bpm is calculated by the next formula: $\frac{NumPeaks}{(\frac{NumFrames}{FPS})} \cdot 60$

Explanation:

- *FPS* – camera's video frame capture rate
- *NumFrames* – the number of last frames saved (arbitrary set to 150 = 5 seconds)
- *NumPeaks* – number of points determined as peaks in the last *NumFrames* frames

The formula itself is not "good enough", as it is very sensitive to changes and doesn't produce a continuous bpm-graph. Thus, we would like to create a better formula using these values.

We use a specific formula: $avg_bpm(i) = \frac{\sum_{i-149}^{i} bpm(i)}{150}$ when an *avg_bpm(i)* represents an arithmetical mean of the last 150 bpm values, calculated in the previous step. This makes the calculations more stable and less sensitive to changes. Yet, we want to improve upon this to get a smoother bpm-graph, that implies better results to the user:

$$avg_avg_bpm(i) = \frac{k}{(k+w)} \cdot avg_avg_bpm(i-1) + \frac{w}{(k+w)} \cdot avg_bpm(i)$$

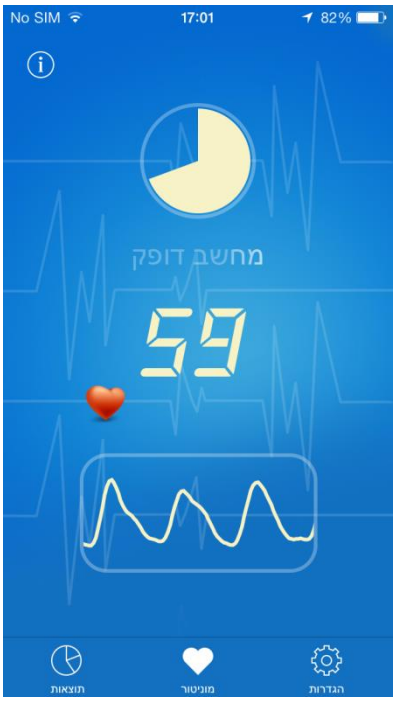
Some explanation first:

- *k* – an index that is initialized after the calibration and updates(*++k*) every iteration of the main loop
- *i* – index of the current frame being processed
- *w* - stands for the weight given to the last avg_bpm value, this way, increasing the weight results in a more sensitive to changes bpm-graph
- We have analyzed and concluded that values between 2 to 3 for w produce good reults

The result - ***avg_avg_bpm(i)*** is the mean over all past averages.

2.6 Final result presentation

At the end, the result is presented in a beautiful UI that is compatible with the latest **iOS 7** appearance. We present you a demonstration of the app (screenshots) on an iPhone 5 running iOS 7:



Monitor Tab



Results Tab

demonstration video: http://youtu.be/dHomdiuAAFw

3. iOS development

We started IOS programming with learning from the iTunes course by Stanford - Developing iOS 7 Apps for iPhone and iPad. (see also the course website)

We learned to use the coreMedia and coreVideo frameworks, used for the video filming with flash. A low capture resolution is used (352x288 pixels) in order to get maximum performance. the frameworks CoreImage and coreGraphics used for the frame processing, converting the array of pixels into floating point value.
The AudioToolbox framework needed to play the beep sound on every heart beat.

In addition, the UIKit framework is necessary framework which is used with all the visual interface implementations. We designed the app to fit the new IOS 7 interface, using the tab bar template. There is the main tab which is used for the measurement of the heart rate, and other two bars used to show the results of previous measurements and the settings of the app.

Other Frameworks:

- **core-plot**: free open source Cocoa plotting framework for OS X and iOS
  the core-plot framework used for demonstrating the heart beat graph in order to give the user proper indication of the measurement.
- **FacebookSDK**
  As its name says, this framework integrated to allow users to login to facebook and share their experiences with the app.

4. The difficulties

In ideal conditions, it should be on par or more accurate then manual measurement. However, a phone is not a proper photoplethysmograph, so the realistic achievable accuracy is likely dependent on a number of factors, including camera performance, proximity of the illuminating LED to the camera lens, and the algorithm used to extract the pulse from the video return.

However, you need to be aware that all photoplethysmographs are *very* sensitive to mechanical movement, so it is essential that the user will hold his finger as still as possible, and avoid moving the phone relative to the finger. In other words, a good Signal-to-Noise Ratio (SNR) is essential to accurately detect the heart rate signal. It can be improved by either reducing the noise or increasing the signal power. In our case, the noise affecting the measurement comes from three sources basically:

- *Image noise:* It originates in the camera sensor. By averaging all the pixels for the brightness calculation, we are filtering out a big part of its spatial component. The band-pass filter that we are applying to the brightness signal is also rejecting much of its time component. However, there will always be some level of this noise in the band of interest.
- *User behavior:* The pressure variations of the fingertip against the camera lens may show up in the signal. If the user has shaky hands, it is better that she uses a finger of the hand holding the phone.
- *Lighting changes:* Since the finger is lit from behind, any change in the light sources or in the scene reflecting that light towards the camera may introduce noise in our pass band. For instance, moving the phone in the air while recording the video sequence may introduce artifacts if the scene parts in the field of view have very different light intensities.

Whereas the noise sources are obviously difficult to control —especially if the system is operated by someone else—, we can increase the signal level by illuminating the finger with a stable light source. If the smartphone has a torch that can be touched with the finger being recorded, it will light the tissues and improve the signal levels.

5. About us

Michael Leybovich - BSc student of Software engineering at the Computer Science Department of the Technion.

Or Maayan - BSc student of Software engineering at the Computer Science Department of the Technion.

Supervisors

Aaron Wetzler - PhD of Electrical engineering at the Technion.